

Cloud Computing Project – Review 1

Multiplayer Robot Arena: programmable robot combat as a cloud service

Project Title

Multiplayer Robot Arena. A browser-based multiplayer arena where every player programs a personal robot in Lua 5.4, deploys it from a persistent SSH development box, and watches the server-authoritative simulation live.

Team Details

Name	Roll number	Branch	Role
Team member 1	21XXXXXXX	CSE	Engine, bots, maps
Team member 2	21XXXXXXX	CSE	API, queue, cloud adapters
Team member 3	21XXXXXXX	CSE	Web client, spectating, docs
Team member 4	21XXXXXXX	CSE	Boxes, provisioning, Lua SDK

Review Details

Review	Review 1
Course	Cloud Computing
Date	DD Month 2026
Faculty guide	Guide name

Introduction

Players sign in, receive one persistent Docker SSH box, and edit `/workspace/main.lua` over SSH. Registration snapshots the script to S3 and starts an agent that keeps one authenticated outbound WebSocket to a Go API. The API runs a deterministic 10 Hz simulation and fans versioned snapshots out to every browser, so spectators and players see the same match in near real time. AWS-compatible services (S3, DynamoDB, SQS) are emulated locally with Floci, keeping the whole stack deployable later to a single VPS.

Problem Statement

Programming-game platforms usually run all robot code on one desktop machine, which gives no cloud architecture to learn from and no shared live view. The project must instead run untrusted player scripts inside per-player boxes, keep all game state server-authoritative, stream the match to any browser, and persist snapshots, match jobs, and results through AWS-compatible services – entirely on student hardware.

Objectives

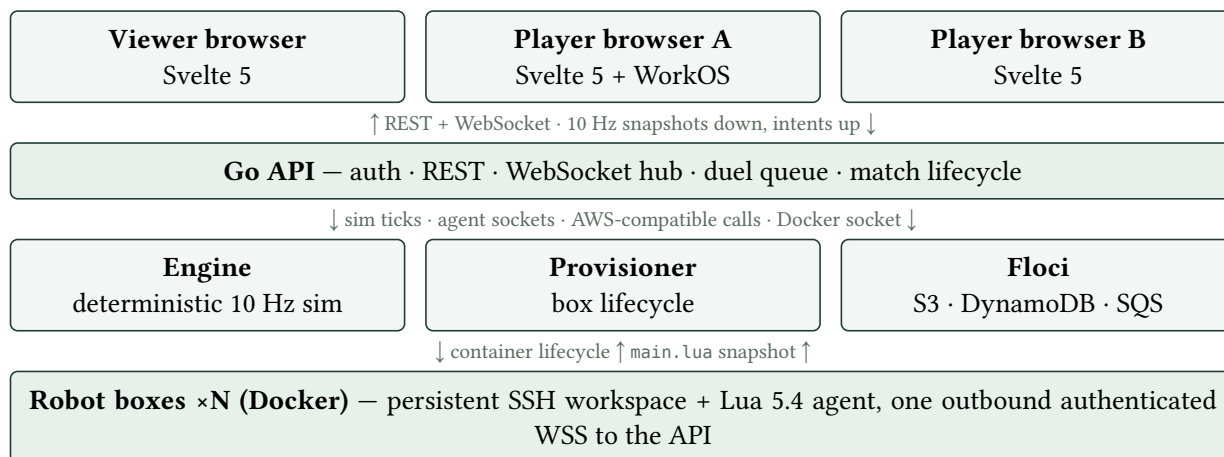
1. A working loop: provision box → edit Lua → register → live synchronized match.
2. Server-authoritative, seed-deterministic engine (no trusted client state).
3. One resource-limited SSH box per player; robot token never reaches the browser.
4. AWS-compatible persistence (S3 snapshots, DynamoDB state, SQS match jobs) behind small adapters.
5. Public spectator surface: live view, history, replays, leaderboard, profiles.

Literature review

- **Robocode / Battlecode / Halite / CodinGame** – programming-game arena concept; ours adds per-player cloud boxes and live spectating.

- **Gambetta, Fast-Paced Multiplayer; Bernier, GDC 2001** — server authority and client interpolation; we apply both at a 10 Hz tick.
- **LocalStack / Floci** — emulate AWS locally so no real account is needed.
- **Agones (Google)** — Kubernetes game-server fleets; deliberately heavier than our single-VPS scope.

Architectural diagram



Modules

Web client (apps/web) · API server (cmd/server, internal/api) · simulation engine and bots (internal/engine) · Lua runtime and SDK (internal/robotlua, sdk/lua) · box provisioning and supervisor (internal/boxes, cmd/provisioner, cmd/box-supervisor) · cloud adapters (internal/cloud) · auth (internal/auth) · data model (internal/model) · curated script templates (internal/scripts).

Status of work

- ✓ End-to-end demo: two boxes, two browsers, synchronized 10 Hz snapshots.
- ✓ Persistent boxes with SSH editing; snapshots to S3; restart-safe stack.
- ✓ Deterministic engine: maps, obstacles, items, weapons, mines, turrets, zone collapse, overtime.
- ✓ Bots: four difficulties × four personalities (aggressive/evasive/camper/mixed), anti-stuck wandering.
- ✓ Duel queue with bot fill, squad 5v5, solo free-for-all; replays, ratings, leaderboard, public spectating.

Next review status

- Obstacle-aware bot movement (grid A* or potential fields).
- Match-creatable arena size picker with clamps.
- Follow-cam and auto-director spectator modes.
- Share preview cards and supply airdrop events.

Conclusion

The Phase 1 loop is demonstrable end to end on local AWS-compatible infrastructure: programmable robots, an authoritative deterministic server, and a watchable, replayable match surface. The next review focuses on smarter navigation and richer viewing, keeping the single-VPS deploy story.

References

1. M. Nelson, “Robocode,” IBM alphaWorks, 2001. robocode.sourceforge.io
2. MIT Battlecode, annual programming competition. battlecode.org
3. Two Sigma, “Halite AI programming challenge,” 2016–2018. halite.io
4. G. Gambetta, “Fast-Paced Multiplayer,” 2014. gabrielgambetta.com
5. Y. Bernier, “Latency Compensating Methods in Client/Server In-game Protocol Design,” GDC 2001.

6. LocalStack — local AWS cloud stack emulator. localstack.cloud
7. Google, “Agones — game server hosting on Kubernetes.” agones.dev
8. Lua 5.4 Reference Manual, PUC-Rio. lua.org/manual/5.4